

PRODUCT REQUIREMENTS DOCUMENT

EstimaPro / EngPro by GUD

Estimation Ticket Management System — KPN Downstream Estimation Control

Version 1.0 · 6 August 2026

Prepared from review of application source code, deployed screenshots, and database schema

Document control

Item	Detail
Document title	Product Requirements Document — EstimaPro / EngPro by GUD
Version	1.0 (baseline, reverse-engineered from as-built system)
Date	6 August 2026
Author	Product analysis based on artefacts in the /docs folder
Source artefacts	1. index (1).html — full application source (2,425 lines, single-file SPA) 2. Screenshot — Manager dashboard (live, 118 tickets, 8 estimators) 3. Screenshot — Site Admin login ("KPN Downstream-Estimation Control") 4. Screenshot — deployed variant "EngPro" at gud-engpro.netlify.app 5. Screenshot — Supabase table editor (8 production tables)
Status	Draft for review

Table of contents

Document control	2
Table of contents	2
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions	4
1.4 Reference environment	4
2. Product overview	6
2.1 Problem statement	6
2.2 Product goals	6
2.3 User roles and personas	6
2.4 Ticket domain model	6
2.5 Evidence of production use	7
3. System architecture and data model	8
3.1 Architecture	8
3.2 High-level system flow	8
3.3 Data model (as observed)	8
4. Functional specification	10
4.1 Application bootstrap & Supabase connection	10
4.2 Workspace setup wizard	11
4.3 Authentication & role-based login	12
4.4 Ticket creation — Admin (New request)	14

4.5 Ticket creation — Site Admin	15
4.6 Manager dashboard.....	15
4.7 Kanban board with filters (Manager / Admin / Member views)	16
4.8 Ticket list (table view)	18
4.9 Ticket detail & update modal (assign / status / deadline / notes).....	18
4.10 Ticket deletion	20
4.11 Member "My tickets" view	21
4.12 Site Admin "My tickets" & deadline management	21
4.13 KPI engine — automatic scoring on completion.....	22
4.14 KPI dashboard — Manager (settings, opening points, bonus/penalty).....	24
4.15 KPI view — Member ("My KPI").....	25
4.16 Settings — workspace, team, site admins, passwords, workspace reset.....	26
4.17 Data synchronisation & live indicator	27
5. Non-functional requirements	29
6. Critical review and findings.....	30
6.1 Security findings.....	30
6.2 Data-integrity findings	30
6.3 Functional defects found in the source	31
6.4 Product and UX gaps.....	31
6.5 Architecture assessment.....	32
6.6 Consistency review of the document set.....	32
7. Recommendations and roadmap	34
7.1 Priority 0 — security remediation (do before anything else)	34
7.2 Priority 1 — data integrity	34
7.3 Priority 2 — product completion	34
7.4 Suggested release plan	34
8. Open questions.....	35
9. Appendix — screen inventory.....	35

1. Introduction

1.1 Purpose

This PRD documents, as a single authoritative reference, the product requirements of EstimaPro (deployed under the name "EngPro" at gud-engpro.netlify.app) — a lightweight estimation ticket management system used by the KPN Downstream estimation team. Because the product was built before any formal specification existed, this document has been reverse-engineered from the as-built source code and live screenshots, then extended with a critical review, high-level and low-level flows for every function, positive and negative user stories, and prioritised recommendations.

1.2 Scope

- In scope: workspace bootstrap and setup, role-based authentication, ticket intake (Admin and Site Admin), kanban board, ticket list, ticket detail management (assign / status / deadline / notes / delete), member workload views, KPI points engine (automatic, opening, manual), settings administration, and data synchronisation.
- Out of scope (not present in the system today): file attachments, e-mail/WhatsApp notifications, reporting exports, drag-and-drop board interaction, mobile native apps, multi-workspace switching in one session.

1.3 Definitions

Term	Meaning
Ticket	A single estimation request (e.g. a tender package to be priced). Numbered EST-001, EST-002... (deployed variant uses #EP-001...).
Workspace	A company/team container in Supabase holding members, tickets, notes and KPI data. Current production workspace: "KPN Downstream-Estimation Control".
Manager	Role with full access: dashboard, board, tickets, KPI administration, settings.
Admin	Role that inputs and manages tickets and views the board and list.
Site Admin	A per-site requester account (own password per site) that submits tickets for its site and sees only its own tickets.
Member (Estimator)	Role that works assigned tickets: updates status, adds progress notes, views team board and personal KPI.
KPI entry	A points transaction for a member in a month: type opening (monthly allowance), auto (system-awarded on ticket completion) or manual (manager bonus/penalty).
Overdue	A ticket whose deadline date is before today and whose status is not Done.

1.4 Reference environment

- Front end: single index.html file — vanilla JavaScript, no framework, no build step; DM Sans / DM Mono fonts; Tabler icons; hosted on Netlify.
- Back end: Supabase (PostgreSQL + PostgREST) accessed directly from the browser with the anon key over the REST API; no server-side application code.
- Production tables observed in Supabase: workspaces, members, site_admins, tickets, notes, tender_notes, kpi_entries, kpi_settings.

2. Product overview

2.1 Problem statement

Estimation requests for the KPN Downstream team arrive through informal channels (WhatsApp, e-mail, verbal requests in meetings). Before this tool, there was no single register of requests, no visibility of estimator workload, no deadline tracking, and no objective performance measure. EstimaPro converts every request into a numbered ticket, routes it through a fixed five-stage workflow, and awards KPI points automatically when work completes on or after its deadline.

2.2 Product goals

- G1 — Every estimation request is captured as a ticket with type, priority, requester, source and deadline within one minute of arrival.
- G2 — Managers see live workload per estimator and can assign/rebalance from one board.
- G3 — Estimators see exactly what is assigned to them, what is urgent and what is overdue.
- G4 — Site personnel can submit and track their own requests without seeing other sites' data.
- G5 — Performance (on-target vs missed deadlines) is scored automatically and transparently per member per month.

2.3 User roles and personas

Role	Persona & primary jobs	Access summary
Manager	Estimation department head. Monitors workload, assigns tickets, sets KPI rules, awards bonuses, administers team and passwords.	Full: Dashboard, Board, Tickets, KPI, Settings. Can edit and delete any ticket.
Admin	Estimation coordinator / document controller. Registers incoming requests, keeps the board tidy.	New request form, Board, All tickets. Can assign, edit, delete tickets. No Settings or KPI admin.
Site Admin	Site engineer at a remote site (e.g. Bontang, Tj. Pura). Raises requests for their site and follows progress.	New request form (site-stamped), own-site ticket list, may change the required-by date and delete own tickets. Cannot see other sites.
Member (Estimator)	Estimator (production users observed: Danu, Halomoan, Luqman, Rully, Sajali, Sumiardi, Wahyu, Yohana). Works tickets, updates status, logs progress notes, tracks personal KPI.	My tickets, view-only team board, My KPI. Can update status/notes only on tickets assigned to them.

2.4 Ticket domain model

Dimension	Values
Status (workflow)	New → In Progress Estimation → In Progress Tender → Done Hold (5 fixed kanban columns; enforced by a database CHECK constraint)
Work type	Project Tender · Ops Tender · Site Instruction / Site Memo · Budgeting Internal
Priority	Urgent · Normal · Low
Request source	WhatsApp · Email · Verbal / meeting

Dimension	Values
Numbering	EST-NNN, zero-padded, sequence = count of existing tickets + 1 (deployed variant: #EP-NNN)

2.5 Evidence of production use

- The dashboard screenshot shows 118 tickets, 61 completed (52% rate), 7 overdue, 10 unassigned across 8 estimators — the system is in real daily use.
- Ticket names are live tender packages (e.g. "Pekerjaan Line Piping ISBL & OSBL Upgrading Refinery Plant 1 POME di PT EUP Tj Pura", "ADD 1000MT SODIUM STORAGE TANK – BONTANG").
- The deployed variant screenshot shows navigation (Kanban, Tickets, KPI, Settings, PROGRAM, New Ticket) and ticket numbering (#EP-001...) that differ from the source file reviewed — the codebase has forked from this file; see Section 6.8.

3. System architecture and data model

3.1 Architecture

The system is a two-tier, serverless architecture. The entire application ships as one static HTML file; all business logic (validation, permissions, ticket numbering, KPI scoring) executes in the browser. Data access goes directly from the browser to the Supabase REST API using a hard-coded project URL, anon key and workspace UUID embedded in the source. Row Level Security is enabled on every table but each table carries a permissive allow_all policy, so the anon key effectively has full read/write access.

3.2 High-level system flow

1. Browser loads index.html from Netlify.
2. init() verifies Supabase connectivity (GET workspaces?select=id&limit=1); on failure an alert is shown and the app stops.
3. The hard-coded workspace row is fetched together with its members and site admins; the login screen is branded with the company name.
4. The user picks a role, optionally selects their name/site, and enters the shared role password (validated client-side against fields of the fetched workspace row).
5. The role-specific screen launches; fetchAll() downloads the full working set (workspace, members, site admins, all tickets, all notes, KPI settings, two years of KPI entries).
6. A polling loop re-runs fetchAll() every 8 seconds and re-renders the active page; a browser visibility listener forces a refresh when the tab regains focus; a header dot indicates Live / Syncing / Offline.
7. All create/update/delete actions issue REST calls (POST/PATCH/DELETE) and then re-fetch the working set.

3.3 Data model (as observed)

Table	Key fields	Notes
workspaces	id (uuid PK), company, subtitle, pw_manager, pw_admin, pw_member, created_at	One row per team. Role passwords stored in plain text.
members	id, workspace_id (FK), name, color (0–8), role (admin member)	Estimators and admins. Referenced elsewhere by name string, not by id.
site_admins	id, workspace_id (FK), site_name, password, color	One row per site; password unique per site (enforced client-side only) and stored/displayed in plain text.
tickets	id, workspace_id (FK), ticket_no, name, type, requestor, priority, status (CHECK on 5 values), assignee (text), deadline (date), source, description, submitted_by, site_name, created_at, updated_at	assignee/site linkage is by name text; renames are cascaded by client-side PATCH calls.
notes	id, ticket_id (FK, cascade delete), author, content, status_at_time, created_at	Progress notes shown newest-first in the ticket modal.
kpi_settings	id, workspace_id, point_opening (default 10), point_on_target (default 3), point_miss_target (default –2)	Table exists in production but is absent from the setup SQL inside the app (schema drift).

Table	Key fields	Notes
kpi_entries	id, workspace_id, member_name, month, year, points, type (opening auto manual), description, ticket_id, ticket_no, created_by	Points ledger. Also absent from the in-app setup SQL.
tender_notes	(schema not visible in reviewed source)	Exists in production Supabase but is never referenced by the reviewed source file — evidence of a newer forked build.

4. Functional specification

Each function below is specified with: an overview, a high-level flow (business view), a low-level flow (as implemented, step-by-step with actual behaviour and edge handling), and positive and negative user stories. Story IDs use the pattern F<n>-P/N<i>.

4.1 Application bootstrap & Supabase connection

Attribute	Detail
Actors	All users (automatic on page load)
Trigger	Opening the application URL
Outcome	Connected session on the branded login screen
Key code	init(), loadConfig(), loadWorkspaceAndShowLogin(), saveConfig() [dormant]

4.1.1 High-level flow

1. User opens the app URL.
2. The app verifies it can reach the database.
3. The team workspace, member list and site list are loaded.
4. The login screen appears, branded with the company name and subtitle.

4.1.2 Low-level flow (as implemented)

1. loadConfig() returns the hard-coded Supabase URL, anon key and workspace UUID (constants HARCODED_URL / HARCODED_KEY / HARCODED_WS); the first-run configuration screen and its 3-step SQL instructions are permanently bypassed.
2. A connectivity probe (GET /rest/v1/workspaces?select=id&limit=1) runs behind a "Connecting..." overlay; on any error a blocking alert "Cannot connect to Supabase" is shown and initialisation aborts.
3. The URL query string is parsed for ?member=<name> (a shareable deep link generated in Settings).
4. The workspace row is fetched by UUID; if no row is returned the setup wizard is shown instead (in practice unreachable because the UUID is fixed).
5. Members (ordered by name) and site admins (ordered by site name) are fetched into module-level state arrays.
6. If the member deep-link matches a member name, the login screen opens pre-set to the Member role with that name pre-selected in the dropdown; the user only types the shared member password.
7. Otherwise the role-selection login screen is shown.

4.1.3 Implementation notes and known issues

- The configuration screen (saveConfig, copySQL) and the ?ws= parameter in generated member links are dead code — retained in the page but unreachable/ignored.
- The setup SQL displayed on the (dormant) config screen omits the kpi_settings, kpi_entries and tender_notes tables that production depends on: a new installation following those instructions would break the KPI module.

4.1.4 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F1-P1	Positive	As an estimator, I want the app to open straight to my company's login screen so that I can sign in without any technical setup.	App loads, shows "KPN Downstream-Estimation Control" branding and the four role buttons within a few seconds; sync overlay disappears.
F1-P2	Positive	As a member, I want my personal link (...?member=Rully) to pre-select my name so that I only need to type the password.	Opening the link lands directly on the Member password step with "Rully" pre-selected.
F1-N1	Negative	As a user with no internet access, I want a clear failure message so that I do not stare at an endless spinner.	Alert "Cannot connect to Supabase. Check your network." is shown; the app does not proceed to login.
F1-N2	Negative	As a user whose deep link contains a name that no longer exists, I want the app to fall back gracefully.	The ?member= value finds no match; the standard role-selection screen is shown instead of an error.
F1-N3	Negative	As the product owner, I do not want database credentials discoverable by anyone who opens View Source.	FAILS today: the Supabase URL, anon key and workspace UUID are readable in the page source (see finding SEC-1).

4.2 Workspace setup wizard

Attribute	Detail
Actors	First-time installer (in practice: developer only — dormant in production)
Trigger	No workspace row found for the configured workspace id
Outcome	Workspace + initial members created in Supabase; user handed to login
Key code	initSetup(), addMemberRow(), wizardNext(), setStep(), goToLogin()

4.2.1 High-level flow

1. Installer names the company (and optional subtitle).
2. Installer adds team members and picks each one's role (Admin or Member).
3. Installer sets three role passwords (Manager, Admin, Member).
4. The workspace is created; a success step links to the login screen.

4.2.2 Low-level flow (as implemented)

1. Step 1 validates that company name is non-empty (alert on failure) and advances.
2. Step 2 seeds two empty member rows; "Add member" appends rows (auto-cycled avatar colour from a 9-colour palette, role dropdown, remove button). On Next, rows with empty names are silently dropped; at least one named member is required (alert otherwise).
3. Step 3 requires each of the three passwords to be at least 4 characters (single combined alert).
4. On Finish: POST /workspaces with company, subtitle and the three plain-text passwords; then one POST /members per wizard row (sequential, no batching, no rollback if one fails mid-loop).
5. Step 4 shows a success message; "Go to login" re-fetches members and shows the login screen.

4.2.3 Implementation notes and known issues

- Because credentials and workspace UUID are hard-coded, this whole wizard is unreachable in the production build; it survives as dead code and would only run if the fixed workspace row were deleted.

4.2.4 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F2-P1	Positive	As an installer, I want a guided 4-step wizard so that a non-technical manager could stand up a new team workspace in minutes.	Completing the four steps creates one workspaces row and N members rows; login screen shows the new company name.
F2-P2	Positive	As an installer, I want to assign Admin vs Member per person during setup so that roles are right from day one.	Role selected per row is persisted on each members record.
F2-N1	Negative	As an installer who skips the company name, I want to be stopped before proceeding.	Alert "Enter company name."; wizard stays on step 1.
F2-N2	Negative	As an installer who removes all member rows, I want to be told a team is required.	Alert "Add at least one member."; wizard stays on step 2.
F2-N3	Negative	As an installer who types a 3-character password, I want the weak password rejected.	Alert "All passwords must be at least 4 characters."; note: 4 characters is itself far below reasonable password policy (finding SEC-3).
F2-N4	Negative	As an installer whose network drops mid-creation, I want no half-created workspace.	FAILS today: members are inserted one-by-one after the workspace; a mid-loop failure leaves a partially populated workspace with no rollback.

4.3 Authentication & role-based login

Attribute	Detail
Actors	Manager, Admin, Site Admin, Member
Trigger	User on login screen; or member deep link
Outcome	Role session established in memory; role app launched
Key code	selectRole(), goToPassword(), doLogin(), backToRole(), logout()

4.3.1 High-level flow

1. User selects one of four roles (Manager, Admin, Admin Site, Member).
2. User provides identity where relevant (name for Admin/Member, site for Site Admin) and the role password.
3. System validates the password and opens the matching workspace view.
4. Logout returns to the login screen and stops background sync.

4.3.2 Low-level flow (as implemented)

1. selectRole() highlights the chosen role card; Continue without a selection alerts "Select a role first."

2. `goToPassword()` switches to the password step and adapts it: Admin/Member show a name dropdown filtered to members with that role; Site Admin shows a site dropdown from `site_admins`; Manager shows password only. Focus is placed in the password field; Enter submits.
3. `doLogin()` compares the typed password in the browser against fields of the already-downloaded workspace row: `pw_manager`, `pw_admin` or `pw_member` respectively; for Site Admin it matches the selected site row's own password field.
4. For Admin/Member/Site Admin, an unselected name/site raises an alert; a wrong password shows the inline red error "Incorrect password. Try again."
5. On success, in-memory session variables are set (`currentRole`, `currentMember` or `currentSiteAdmin`) and the corresponding launch routine runs (`launchManager` / `launchAdmin` / `launchSiteAdmin` / `launchMember`).
6. There is no session persistence: a browser refresh returns the user to the login screen. `logout()` clears state, stops the polling timer and shows the login screen.

4.3.3 Implementation notes and known issues

- Authentication is entirely client-side theatre: because RLS is `allow_all`, the REST API honours reads/writes with or without a "login". Passwords gate the UI only.
- All members share one password (identity is chosen from a dropdown), so actions are attributable only by trust, not by authentication.

4.3.4 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F3-P1	Positive	As a manager, I want a single manager password so that I can open the full dashboard from any device.	Correct <code>pw_manager</code> opens the Manager screen with Dashboard active.
F3-P2	Positive	As an estimator, I want to pick my name and enter the shared member password so that my session shows my tickets and my KPI.	Member screen opens with the user's name in the header chip and "<name>'s tickets" as the page title.
F3-P3	Positive	As a site engineer, I want to pick my site and use my site's own password so that other sites cannot act as us.	Site Admin screen opens stamped with the site name; site password of another site is rejected.
F3-N1	Negative	As a user who mistypes the password, I want an inline error and another attempt.	Red "Incorrect password. Try again." appears; no lockout, unlimited retries (see finding SEC-3: no rate limiting or lockout exists).
F3-N2	Negative	As a member who forgets to select a name, I want to be prompted before submission.	Alert "Select your name." (or "Select your site."), login not attempted.
F3-N3	Negative	As the security owner, I do not want role passwords delivered to the browser before login.	FAILS today: the workspace row including all three plain-text passwords is fetched at page load and is visible in DevTools network traffic to anyone (finding SEC-2).
F3-N4	Negative	As a user whose session is idle overnight, I expect to be re-authenticated.	PARTIALLY MET by accident: refresh loses the session (no persistence), but an open tab stays authenticated indefinitely with no timeout.

4.4 Ticket creation — Admin (New request)

Attribute	Detail
Actors	Admin (also reachable by Manager via code path submitTicket)
Trigger	"New request" tab → Submit request
Outcome	New ticket row with auto number EST-NNN, status New
Key code	submitTicket('admin'), clearAdminForm()

4.4.1 High-level flow

1. Admin fills in the request: work name, requested by, work type, priority, required-by date, optional assignee, request source, description.
2. Admin submits; the system generates the next ticket number and saves the ticket.
3. A confirmation toast shows the new ticket number and the form clears.

4.4.2 Low-level flow (as implemented)

1. Mandatory-field gate: name, type and priority must be present, otherwise alert "Fill in work name, type, and priority." (date, requester, assignee, source, description are optional).
2. The app fetches all existing ticket_no values for the workspace and computes the next number as count + 1, formatted EST-<zero-padded 3 digits>.
3. POST /tickets with status "New", submitted_by = the admin's member name, and null for any blank optional field.
4. fetchAll() refreshes the working set; the form is cleared; toast "Ticket EST-NNN created".
5. On any REST failure the loading overlay closes and an error toast with the raw error message is shown; the form is not cleared.

4.4.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F4-P1	Positive	As an admin, I want to log a WhatsApp request as a ticket in under a minute so that nothing gets lost.	Submitting with name/type/priority creates the ticket; toast confirms "Ticket EST-119 created"; ticket appears in New column on the board.
F4-P2	Positive	As an admin, I want to assign the estimator at intake when the owner is obvious so that work starts immediately.	Ticket created with assignee set; it appears in that member's "My tickets" on their next sync (≤8 s).
F4-N1	Negative	As an admin who forgets the work type, I want to be blocked with a clear message.	Alert lists the three required fields; nothing is saved; entered values are preserved.
F4-N2	Negative	As an admin submitting while offline, I want my input preserved for retry.	Error toast shown; form content remains; resubmission after reconnect succeeds.
F4-N3	Negative	As the data owner, I never want two tickets to share a number.	FAILS under concurrency: two users submitting within the same window both read the same count and both create e.g. EST-119 (finding DATA-1). Ticket numbers are also recycled if tickets are deleted (count-based, not sequence-based).

4.5 Ticket creation — Site Admin

Attribute	Detail
Actors	Site Admin
Trigger	"New request" tab on the Site Admin screen
Outcome	New unassigned ticket stamped with the site (site_name and submitted_by = site name)
Key code	submitSiteTicket(), clearSiteForm()

4.5.1 High-level flow

1. Site engineer fills in the request form (same fields as Admin intake but with no "Assign to" control).
2. On submit the ticket is created unassigned, stamped with the site name, and enters the New column for the estimation team to triage.

4.5.2 Low-level flow (as implemented)

1. Same mandatory-field gate as F4 (name, type, priority).
2. Ticket number generated by the same count+1 EST-NNN logic (shared numbering pool with Admin intake — same race risk).
3. POST /tickets with assignee null, status "New", submitted_by = site name, site_name = site name; these two stamps are what scope the site's "My tickets" view.
4. fetchAll(), clear form, toast with the ticket number.

4.5.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F5-P1	Positive	As a site engineer at Bontang, I want to raise an estimation request from site so that I do not have to chase the head office via WhatsApp.	Ticket created with site stamp; appears in the site's "My tickets" list and in the central board's New column.
F5-P2	Positive	As a site engineer, I want my submissions to be automatically unassigned so that the estimation manager controls dispatching.	New ticket shows "Waiting" as assignee in the site list until a manager/admin assigns it.
F5-N1	Negative	As a site engineer omitting priority, I want to be stopped with a clear message.	Alert shown; no ticket created.
F5-N2	Negative	As a manager, I never want a site to submit under another site's identity.	Site identity comes from the authenticated site session, not from a form field — cannot be forged through the UI (though the open REST API undermines this; SEC-2).

4.6 Manager dashboard

Attribute	Detail
Actors	Manager

Attribute	Detail
Trigger	Login as Manager, or Dashboard tab
Outcome	Live operational overview
Key code	renderDashboard(), isOverdue()

4.6.1 High-level flow

1. Manager opens the dashboard and reads: headline stats, status breakdown, type breakdown, per-estimator workload, recent tickets.
2. From "Recent tickets" the manager can jump straight into a ticket's edit modal.

4.6.2 Low-level flow (as implemented)

1. Headline stats computed in memory from allTickets: Total, Completed (with % rate), Overdue (deadline < today, not Done — highlighted red), Unassigned (no assignee, not Done — highlighted amber).
2. Status grid: one tinted mini-card per workflow state with count. Type grid: one card per work type.
3. Workload grid: one card per member (role member only), showing avatar, active load count, a load bar (active ÷ total assigned, in the member's avatar colour), and Done/Active counters.
4. Recent tickets: top 6 by created_at desc with number, name, type badge, status badge, assignee (or red "Unassigned") and an edit shortcut opening the shared ticket modal (F9).
5. Re-rendered every 8 s by the polling loop while the dashboard tab is active.

4.6.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F6-P1	Positive	As a manager, I want an at-a-glance count of overdue and unassigned tickets so that I know where to intervene first.	Overdue and Unassigned tiles show correct counts in red/amber; screenshot evidence: 7 overdue, 10 unassigned of 118.
F6-P2	Positive	As a manager, I want workload per estimator so that I can balance assignments.	Each estimator card shows active vs done counts matching board contents (e.g. Rully 15 active / 21 done).
F6-N1	Negative	As a manager with a brand-new workspace, I want sensible empty states rather than errors.	Zero tickets renders 0-value stats, "No members yet." and an empty-inbox placeholder for recent tickets.
F6-N2	Negative	As a manager, I want completion rate to be meaningful even with zero tickets.	Division-by-zero is guarded; rate shows 0%.

4.7 Kanban board with filters (Manager / Admin / Member views)

Attribute	Detail
Actors	Manager, Admin (editable); Member (view + own-ticket edit via modal)
Trigger	Board / Team board tab
Outcome	Five-column status board, filterable by member, status and type

Attribute	Detail
Key code	renderBoardEl(), getFilteredTickets(), resetBoardFilter(), renderBoard/renderAdminBoard/renderMemberBoard

4.7.1 High-level flow

1. User opens the board and sees five fixed columns (New, In Progress Estimation, In Progress Tender, Done, Hold) with per-column counts.
2. User narrows the view with three dropdown filters (member incl. Unassigned, status, type); a counter shows "X of Y tickets"; Reset restores all.
3. Clicking any card opens the ticket detail modal (F9).

4.7.2 Low-level flow (as implemented)

1. getFilteredTickets() applies member ("__unassigned" matches null assignee), status and type filters over the in-memory ticket array.
2. When a status filter is active, only that single column is rendered; otherwise all five columns render.
3. Each card shows: ticket number (mono), name, type + priority badges, an italic excerpt (max 55 chars) of the latest progress note if any, the due date element (red + "Overdue" pill when overdue), and assignee name + avatar or a red "unassigned" marker. Cards with notes get a blue left border; overdue cards get a red border and tinted background.
4. Filter dropdowns are re-populated from the member list on every render, preserving the current selection.
5. There is no drag-and-drop: status changes happen only through the modal.
6. The member-role board is captioned "View-only overview" but uses the same modal; edit rights inside the modal are governed by F9's permission matrix (member can edit only tickets assigned to them).

4.7.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F7-P1	Positive	As a manager, I want to filter the board to one estimator so that I can run a 1-on-1 review of their queue.	Selecting the member shows only their cards; count reads "N of 118 tickets"; Reset restores everything.
F7-P2	Positive	As an admin, I want unassigned tickets isolated in one click so that I can dispatch them.	The "Unassigned" filter option lists exactly the tickets with no assignee.
F7-P3	Positive	As any user, I want overdue cards to be unmissable so that late work gets attention.	Overdue cards render with red border, tinted background and an "Overdue" pill.
F7-N1	Negative	As a user filtering a combination with no matches, I want an empty state, not a broken column.	Column renders the "Empty" placeholder; count shows "0 of 118 tickets".
F7-N2	Negative	As an estimator on the team board, I must not be able to edit a colleague's ticket.	Opening a colleague's card shows read-only details with the message that only the assignee can update it (note rendering defect BUG-1 in that message).

4.8 Ticket list (table view)

Attribute	Detail
Actors	Manager, Admin
Trigger	Tickets / All tickets tab
Outcome	Full sortable-by-recency table with the same filters as the board
Key code	renderTableEl(), renderTable(), renderAdminTable(), resetTableFilter()

4.8.1 High-level flow

1. User opens the list of all tickets (newest first) with columns: ID, Name, Type, Priority, Status, Assignee, Due, Notes count.
2. User filters by member/status/type and opens any ticket via its Edit button.

4.8.2 Low-level flow (as implemented)

1. Same getFilteredTickets() pipeline as the board; rows render badges identical to card badges; overdue due-dates render red with the Overdue pill; the Notes column shows a count badge.
2. canEdit=true for both Manager and Admin variants, adding an Edit button per row that opens the shared modal (F9).
3. Empty result renders a single "No tickets" row; the "X of Y tickets" counter behaves as on the board.
4. No column sorting, no text search, no pagination — the entire ticket set renders in one table (see GAP-1/GAP-2).

4.8.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F8-P1	Positive	As an admin, I want a flat list of every ticket so that I can audit and clean up records the board makes hard to scan.	Table lists all tickets newest-first with accurate badges and note counts; Edit opens the modal.
F8-P2	Positive	As a manager, I want the same filters on list and board so that I do not relearn the UI.	Member/status/type dropdowns behave identically in both views.
F8-N1	Negative	As a manager searching for one ticket by name, I currently have no search box.	GAP: user must visually scan or use browser Ctrl-F within the rendered page (GAP-1).
F8-N2	Negative	As a user on a workspace with thousands of tickets, I want the list to stay responsive.	RISK: no pagination or virtualisation; every poll re-renders the full table (GAP-2).

4.9 Ticket detail & update modal (assign / status / deadline / notes)

Attribute	Detail
Actors	Manager, Admin (full); assigned Member (status + note); others (read-only)
Trigger	Clicking a card, a row Edit button, or a recent-ticket shortcut
Outcome	Ticket updated; optional progress note appended; auto-KPI may fire
Key code	openModal(), saveModal(), closeModal()

4.9.1 High-level flow

1. User opens a ticket and reviews its full details (metadata, description, chronological progress notes).
2. Depending on role, the user reassigns it, changes status, moves the required-by date, and/or writes a progress note.
3. Saving persists the changes; if the ticket just became Done, KPI points are awarded automatically to the assignee (F13).

4.9.2 Low-level flow (as implemented)

1. Permission matrix computed at open: canEditStatus = Manager | Admin | (Member AND ticket assigned to them); canAssign, canChangeDate, canDelete = Manager | Admin.
2. Read-only visitors (e.g. a member viewing someone else's ticket) see details plus a notice that only the assignee may update it.
3. Editable form: Assign to (member dropdown incl. Unassigned) if permitted; Status (5 options); Required-by date with an Overdue pill next to the label when late; free-text "Add progress note".
4. saveModal(): PATCH /tickets with assignee, status, updated_at (client clock) and deadline (when the date control was present).
5. Auto-KPI trigger: if new status is Done, previous status was not Done, and an assignee exists, autoKpiEntry() runs before the note insert (details in F13).
6. If note text was entered, POST /notes with author = current member name (or literal "Manager"), content, and status_at_time = the newly chosen status.
7. fetchAll() then re-renders whichever page is active for the current role; toast "Saved"; failures toast the raw error and keep the modal open.

4.9.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F9-P1	Positive	As a manager, I want to assign a new ticket to an estimator and set its status in one dialog so that triage is a single step.	Choosing assignee + status and saving updates the card within the same render; the member sees it on next 8 s poll.
F9-P2	Positive	As an estimator, I want to log "waiting for vendor quote" against my ticket so that everyone sees why it is parked.	Note appears newest-first in the modal with author, timestamp and the status at the time; board card shows the excerpt.
F9-P3	Positive	As an estimator, I want marking my ticket Done to automatically credit my KPI so that scoring needs no admin work.	On Done transition an auto entry (+point_on_target or point_miss_target) is written and visible in My KPI.
F9-N1	Negative	As an estimator, I must not be able to modify tickets assigned to colleagues.	Modal renders read-only; no Save control is present.
F9-N2	Negative	As a manager whose save hits a network error, I want my edits preserved for retry.	Error toast shows; modal stays open with entered values.
F9-N3	Negative	As a user viewing an unassigned ticket read-only, I want a human message about who can edit it.	FAILS today: the message prints the literal text "\${t.assignee 'nobody'}" instead of the assignee name — template-escaping defect (BUG-1).
F9-N4	Negative	As a manager, I do not want KPI double-awards if two sessions mark the same ticket Done simultaneously.	RISK: the duplicate check reads the local cache only; concurrent saves can both pass it (DATA-4).

4.10 Ticket deletion

Attribute	Detail
Actors	Manager, Admin; Site Admin (for own-site tickets, via a dead code path — see notes)
Trigger	Delete button inside the ticket modal
Outcome	Ticket and all its notes permanently removed
Key code	deleteTicket(), confirmDeleteTicket()

4.10.1 High-level flow

1. User clicks Delete in the ticket modal.
2. A dedicated red confirmation dialog warns that the ticket and all progress notes will be permanently removed.
3. On confirmation the ticket disappears everywhere.

4.10.2 Low-level flow (as implemented)

1. deleteTicket() swaps the modal content for a confirmation panel showing the ticket number and an irreversibility warning.
2. confirmDeleteTicket(): DELETE /notes?ticket_id=eq.<id> first, then DELETE /tickets?id=eq.<id> — two independent calls, not a transaction (the notes FK has ON DELETE CASCADE anyway, making the first call redundant but harmless).
3. fetchAll() refresh, toast "Ticket deleted", and re-render of the active page for manager/admin/siteadmin roles.
4. KPI entries referencing the ticket are NOT removed or reversed: points awarded for a deleted ticket persist in the ledger (kpi_entries keeps ticket_no text).

4.10.3 Implementation notes and known issues

- Ticket numbers are count-derived, so deleting tickets causes future numbers to collide with existing ones (e.g. 118 tickets, delete one → next ticket becomes EST-118, a duplicate of the survivor).

4.10.4 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F10-P1	Positive	As an admin, I want to delete a duplicate ticket after an intake mistake so that the register stays clean.	Confirmation dialog → ticket and notes gone; board, table and dashboard counts all drop by one.
F10-P2	Positive	As a manager, I want an explicit, scary confirmation before deletion so that a mis-click cannot destroy work.	Two-step delete with a red dialog naming the ticket number.
F10-N1	Negative	As a manager, I want deleting a completed ticket to not silently distort KPI history.	PARTIAL: KPI points survive deletion (arguably correct), but there is no way to see the deleted ticket behind the ledger entry, and no reversal option (GAP-6).
F10-N2	Negative	As a compliance owner, I want deletions logged.	GAP: no audit trail of who deleted what and when.

4.11 Member "My tickets" view

Attribute	Detail
Actors	Member (estimator)
Trigger	Member login / My tickets tab
Outcome	Personal queue with urgency and overdue surfacing
Key code	renderMyTickets(), memCard()

4.11.1 High-level flow

1. Estimator sees personal stats (assigned to me / active / overdue).
2. If anything is overdue, a red banner lists every overdue ticket by number and name.
3. Urgent-and-active tickets are pinned in an "Urgent — action needed" section; everything else follows.
4. Clicking any card opens the update modal (F9) with member-level rights.

4.11.2 Low-level flow (as implemented)

1. Tickets filtered by assignee === current member name (string match).
2. Stats: total mine, active (status ≠ Done), overdue (isOverdue).
3. Overdue banner concatenates "EST-NNN · name" for each overdue ticket.
4. Cards show number, name, status badge, type + priority badges, full description, the latest note excerpt (80 chars), due-date element and note count; urgent cards carry a red left border, overdue cards a full red border.
5. Empty state "No tickets assigned to you yet" when the member has nothing.

4.11.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F11-P1	Positive	As an estimator starting my day, I want my urgent and overdue work pinned on top so that prioritisation is automatic.	Urgent active tickets render first under a red heading; the overdue banner enumerates late items.
F11-P2	Positive	As an estimator, I want each card to carry the last progress note so that I recall context without opening the ticket.	Latest note excerpt (up to 80 chars, italic) is shown on the card.
F11-N1	Negative	As a new team member with no assignments, I want a friendly empty state.	Inbox placeholder with "No tickets assigned to you yet".
F11-N2	Negative	As an estimator whose name the manager just edited, I do not want to lose my ticket linkage.	HANDLED with caveats: renaming a member cascades a PATCH over tickets matching the old assignee string; but tickets updated concurrently or notes' author fields keep the old name (DATA-2).

4.12 Site Admin "My tickets" & deadline management

Attribute	Detail
Actors	Site Admin
Trigger	My tickets tab on the Site Admin screen
Outcome	Site-scoped progress visibility; intended: due-date extension
Key code	renderSiteAdminTickets(), openSiteModal() [orphaned], saveSiteDeadline()

4.12.1 High-level flow

1. Site engineer opens "My tickets" and sees site-scoped stats (total / active / overdue) and a table of every ticket the site submitted, with assignee ("Waiting" until dispatched) and due dates.
2. Intended: clicking a ticket opens a site modal to view notes and extend/move the required-by date, or delete the request.

4.12.2 Low-level flow (as implemented)

1. Scope filter: tickets where site_name or submitted_by equals the current site name.
2. Stats and table render read-only columns: ID, Name, Type, Priority, Status, Assignee, Due (with overdue styling).
3. openSiteModal() implements a full site-facing detail view — ticket metadata, progress notes, a deadline date field saved by saveSiteDeadline() (PATCH deadline + updated_at), and a Delete button — but NO ROW IN THE TABLE INVOKES IT: rows are rendered without a click handler, so the modal is unreachable dead code (BUG-2).
4. The page subtitle still promises "Click to view progress or update date."

4.12.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F12-P1	Positive	As a site engineer, I want to see whether my request has been picked up and by whom so that I can plan site work.	Table shows assignee once dispatched; "Waiting" in amber beforehand; status badge tracks workflow.
F12-P2	Positive	As a site engineer, I want to extend the required-by date when site priorities shift so that estimators are not chasing a stale deadline.	INTENDED behaviour via the site modal; currently unreachable (BUG-2).
F12-N1	Negative	As a site engineer, I must not see or affect other sites' tickets.	List is filtered to my site only; the UI offers no cross-site access (REST API openness notwithstanding — SEC-2).
F12-N2	Negative	As a site engineer following the on-screen hint to "click to update the date", I expect a click to do something.	FAILS today: rows have no click handler; nothing happens (BUG-2).

4.13 KPI engine — automatic scoring on completion

Attribute	Detail
Actors	System (triggered by any user completing a ticket via F9)
Trigger	Ticket status transitions to Done with an assignee

Attribute	Detail
Outcome	One auto KPI entry: +point_on_target (default +3) or point_miss_target (default -2)
Key code	autoKpiEntry(), kpiSettings defaults in fetchAll()

4.13.1 High-level flow

1. When an assigned ticket is marked Done, the system decides on-target vs missed by comparing completion time against the ticket deadline.
2. The assignee is credited (or debited) in the current month's ledger with a descriptive entry referencing the ticket.

4.13.2 Low-level flow (as implemented)

1. Guard clauses: skip if no KPI settings or no assignee; skip if the local cache already holds an auto entry for this ticket (one award per ticket, ever).
2. Scoring: if the ticket has a deadline and now > deadline 23:59:59 local time → points = point_miss_target, description "Missed deadline"; otherwise (on time, or no deadline at all) → point_on_target, "On target".
3. Entry is written to the month/year of COMPLETION (client clock), with type "auto", the ticket number and name in the description, ticket_id/ticket_no references and created_by "System"; then the KPI cache is refreshed.
4. Failures log to console only — the ticket save succeeds even if the KPI write fails (silent divergence).
5. Reopening a Done ticket and completing it again does NOT re-award (the one-per-ticket guard); there is also no reversal when a Done ticket is reopened.

4.13.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F13-P1	Positive	As an estimator who finishes before the deadline, I want on-target points credited automatically so that good delivery is rewarded without paperwork.	Auto entry +3 (configurable) appears in the ledger with "On target — EST-NNN <name>".
F13-P2	Positive	As a manager, I want late completions to cost points automatically so that deadlines carry weight.	Completion after deadline writes the negative miss-target points with "Missed deadline" description.
F13-N1	Negative	As a manager, I do not want a member farming points by toggling a ticket Done repeatedly.	One auto entry per ticket ever; re-completions award nothing.
F13-N2	Negative	As an estimator whose KPI write fails silently, I want to know my points were not recorded.	FAILS today: error goes to the browser console only; ticket shows Done, ledger shows nothing (BUG-5).
F13-N3	Negative	As a manager, I want an award reversed if a ticket is reopened because it was not actually finished.	GAP: no reversal mechanism; the original entry stands and can never be re-earned (GAP-6).
F13-N4	Negative	As a manager, I expect a ticket with no deadline to be scored consciously, not silently as on-target.	DESIGN NOTE: deadline-less tickets always score full on-target points — an incentive to leave deadlines blank.

4.14 KPI dashboard — Manager (settings, opening points, bonus/penalty)

Attribute	Detail
Actors	Manager
Trigger	KPI tab
Outcome	Monthly leaderboard, points administration, transaction history
Key code	renderKPI(), renderKPIMonth(), openKPISettings()/saveKPISettings(), openAddOpening()/saveOpeningPoints(), openAddBonus()/saveBonus()

4.14.1 High-level flow

1. Manager selects a year (current or previous) and a month; sees per-month team totals on the month strip, then per-member score cards for the selected month sorted best-first, with the leader crowned.
2. Manager configures the three scoring parameters, distributes monthly opening points, and records ad-hoc bonuses or penalties with reasons.
3. A transaction table lists every ledger entry for the selected month.

4.14.2 Low-level flow (as implemented)

1. Year selector offers current and previous year (matching the two-year fetch window); month strip shows the 12 team totals, dims future months, colours totals green/red.
2. Member cards: monthly total (green positive / red negative), breakdown across Opening / Tickets (auto) / Bonus (manual), a per-member "Bonus" shortcut, and a trophy + amber top border for the leader when their total is positive.
3. KPI Settings modal edits point_opening, point_on_target, point_miss_target; saved by PATCH when a settings row exists, otherwise POST (upsert emulation).
4. Opening points modal lists every member with a numeric input pre-filled from the default or their existing opening entry (marked "Set"); saving deletes any existing opening entry per member and inserts a fresh one — delete-then-insert, not atomic.
5. Bonus modal requires a member and a non-zero integer; positive = bonus, negative = penalty; description optional (defaults to "Manual adjustment"); created_by "Manager".
6. History table renders member, type badge, signed points pill, description, ticket number and date for all entries in the selected month.

4.14.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F14-P1	Positive	As a manager, I want a monthly leaderboard so that performance conversations are grounded in shared numbers.	Cards sort by monthly total; leader carries the trophy; breakdown separates allowance, earned and discretionary points.
F14-P2	Positive	As a manager, I want to grant every member their monthly opening points in one dialog so that the month starts consistently.	Opening modal lists all members with defaults; saving writes one opening entry each; re-running replaces rather than duplicates.
F14-P3	Positive	As a manager, I want to award +5 for exceptional tender work with a written	Manual entry appears in the ledger with description and "Manager" as creator.

ID	Type	User story	Acceptance criteria / expected behaviour
		reason so that discretionary points are transparent.	
F14-N1	Negative	As a manager entering 0 or blank bonus points, I want to be stopped.	Alert "Enter a non-zero point value."; nothing saved.
F14-N2	Negative	As a manager who forgot to pick a member for a bonus, I want a clear prompt.	Alert "Select a member."
F14-N3	Negative	As a manager reviewing history, I want manual entries visually distinct.	PARTIAL: the manual-type badge references CSS class b-review which does not exist, so it renders unstyled/grey (BUG-3).
F14-N4	Negative	As a manager, I want to review KPI for years older than last year.	GAP: fetch window and year selector are hard-limited to the current and previous year (GAP-7).

4.15 KPI view — Member ("My KPI")

Attribute	Detail
Actors	Member
Trigger	My KPI tab
Outcome	Personal month/year points transparency
Key code	renderMemberKPI()

4.15.1 High-level flow

1. Estimator opens My KPI and sees this month's total, on-target count and missed count; a 12-month strip of personal totals; and the full personal transaction history for the year.

4.15.2 Low-level flow (as implemented)

1. Current-month stats: total points (red when negative), count of positive auto entries ("On target"), count of negative auto entries ("Missed") — current month only.
2. Monthly strip: 12 cells for the current year, current month highlighted, future months dimmed with an em-dash.
3. History table: all of the member's entries for the current calendar year, newest first, with month, type, signed points, description, ticket and date.
4. View is strictly read-only; no year selector (current year only).

4.15.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F15-P1	Positive	As an estimator, I want to see exactly which tickets earned or cost me points so that the score feels fair and auditable.	Each auto entry lists the ticket number and name in its description; totals reconcile with the manager view.
F15-P2	Positive	As an estimator, I want a month-by-month picture of my year so that I can see my trend.	The 12-cell strip shows signed totals per month; current month highlighted.

ID	Type	User story	Acceptance criteria / expected behaviour
F15-N1	Negative	As an estimator with no entries yet, I want an encouraging empty state, not an error.	Trophy placeholder with "No KPI entries yet this year".
F15-N2	Negative	As an estimator in January, I want to review last year's history.	GAP: the member view is fixed to the current calendar year; December's record disappears from view at new year (GAP-7).

4.16 Settings — workspace, team, site admins, passwords, workspace reset

Attribute	Detail
Actors	Manager only
Trigger	Settings tab
Outcome	Administration of workspace identity, people, sites and credentials
Key code	saveWorkspace(), renderSettingsMembers(), saveMember()/updateMember()/deleteMember(), copyMemberLink(), renderSettingsSiteAdmins(), saveSiteAdmin()/updateSiteAdmin()/deleteSiteAdmin(), savePasswords(), startNewWorkspace()/doNewWorkspace()

4.16.1 High-level flow

1. Manager edits company name/subtitle; adds, renames, recolours, re-roles or removes team members; copies per-member login deep links; manages site-admin accounts (name, unique password, colour); rotates the three role passwords; or spins up a fresh workspace.

4.16.2 Low-level flow (as implemented)

1. Workspace: PATCH company/subtitle; header logo text updates immediately; empty company name is rejected.
2. Members: Add modal (name, role, one of 9 avatar colours) → POST. Edit modal → PATCH; if the name changed, a follow-up PATCH rewrites tickets.assignee from old to new name (notes.author and kpi_entries.member_name are NOT rewritten — historical records keep the old name, and KPI totals silently detach from the renamed member; DATA-2). Delete (confirm dialog) first unassigns the member's tickets (assignee → null) then deletes the member row.
3. Member links: each row shows and copies <base-url>?member=<name>&ws=<workspace-id>; the ws parameter is ignored by the loader (BUG-6).
4. Site admins: Add modal enforces name present, password ≥ 4 chars, and password uniqueness across sites (because login identifies the site by dropdown + its own password). Edit allows rename (cascades tickets.site_name and submitted_by via PATCH), password change (uniqueness re-checked) and colour. Delete confirms with "their tickets stay but become unowned". Site passwords are displayed in clear text in the list (SEC-4).
5. Passwords: any of the three fields left blank keeps the current value; ≥4-char rule per filled field; PATCH workspaces; fields cleared after save; toast "No changes" when all blank.
6. New workspace: confirmation modal explains current data stays intact; confirming clears in-memory state and opens the setup wizard (F2) — with hard-coded workspace loading, the newly created workspace will not load on next visit (BUG-7).

4.16.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F16-P1	Positive	As a manager onboarding a new estimator, I want to add them with a role and colour and send them a personal login link so that they are productive in minutes.	Member row appears with copyable deep link; link opens Member login with the name pre-selected.
F16-P2	Positive	As a manager, I want renaming a member to keep their ticket assignments intact.	tickets.assignee values are rewritten to the new name in the same operation (with the caveats in DATA-2).
F16-P3	Positive	As a manager, I want each site to have its own password so that offboarding one site does not disturb others.	Site rows carry independent passwords; uniqueness enforced on create and edit.
F16-N1	Negative	As a manager adding a site with a password another site already uses, I want to be stopped (logins would be ambiguous).	Alert "Password already used by another site. Use a unique password."
F16-N2	Negative	As a manager, I want removing a member to leave their tickets recoverable rather than deleted.	Confirm dialog; tickets become Unassigned and surface in the unassigned filter/tile.
F16-N3	Negative	As a security owner, I do not want site passwords readable on screen by anyone standing behind the manager.	FAILS today: passwords print in plain text in the Site Admins list (SEC-4).
F16-N4	Negative	As a manager adding two members with identical names, I expect the system to prevent ambiguity.	FAILS today: no duplicate-name check; since all linkage is by name string, two "Andi" records corrupt assignment scoping (DATA-2).

4.17 Data synchronisation & live indicator

Attribute	Detail
Actors	System (all authenticated roles)
Trigger	Login; every 8 seconds thereafter; tab becoming visible
Outcome	All clients converge on the same data without manual refresh
Key code	fetchAll(), startPolling()/stopPolling(), setSyncStatus(), visibilitychange handler, rerenderActive()

4.17.1 High-level flow

1. After login the app keeps itself current automatically; a coloured dot in the top bar shows Live (green), Syncing (amber pulse) or Offline (red).
2. Changes made by any user become visible to everyone else within about 8 seconds.

4.17.2 Low-level flow (as implemented)

1. fetchAll() re-downloads, in sequence: the workspace row (so password/name changes propagate), members, site admins, ALL tickets (newest first), ALL notes for those tickets (single request using an in.(id,id,...) filter over every ticket id), KPI settings (with client-side defaults 3/-2/10 if absent) and two years of KPI entries.

2. `startPolling()` runs `fetchAll` + a role-specific re-render every 8 s; only the currently visible page is re-rendered; the timer is a single global handle cleared on logout.
3. Any fetch error flips the indicator to Offline; the next successful cycle flips it back to Live.
4. A `visibilitychange` listener triggers an immediate fetch + re-render when the tab regains focus.
5. There is no delta sync, no ETag/If-None-Match, and no Supabase Realtime subscription: cost grows linearly with tickets × polling clients; the notes in.() URL grows with every ticket and will eventually exceed URL length limits (GAP-2).

4.17.3 User stories

ID	Type	User story	Acceptance criteria / expected behaviour
F17-P1	Positive	As a manager watching the board during a stand-up, I want new tickets and status moves to appear on their own so that the board is trustworthy live.	Changes by others appear within ≤8 s without interaction; dot stays green.
F17-P2	Positive	As a user returning to a background tab, I want it to catch up instantly.	Tab focus triggers an immediate refresh ahead of the next poll.
F17-N1	Negative	As a user on a train with flaky connectivity, I want to know when what I see may be stale.	Dot turns red "Offline" on fetch failure and recovers automatically.
F17-N2	Negative	As a user editing in the modal while a poll lands, I do not want my open dialog overwritten.	HANDLED implicitly: polls re-render pages, not the open modal; but the modal's data may be stale by save time — last-writer-wins with no conflict detection (DATA-6).

5. Non-functional requirements

#	Category	Requirement (target)	Current state
NFR-1	Security	Credentials never shipped in client code; server-enforced authorisation per role and workspace; passwords hashed; per-user identity.	NOT MET — anon key + workspace UUID hard-coded; allow_all RLS; plain-text shared passwords compared client-side.
NFR-2	Performance	Board/table interactive < 2 s at 1,000 tickets; sync payloads scale sub-linearly (delta or realtime).	PARTIAL — fine at ~120 tickets; full-refresh polling every 8 s per client will degrade with growth.
NFR-3	Availability	Static hosting + managed DB; graceful offline indication.	MET for current scale (Netlify + Supabase; offline dot present).
NFR-4	Usability	Single-page, role-scoped UIs; overdue/urgent visually unmissable; mobile-friendly.	LARGELY MET — clean role UIs and overdue treatment; some grids not fully responsive; no accessibility audit.
NFR-5	Data integrity	Unique ticket numbers; referential integrity for people links; atomic multi-step writes.	NOT MET — count-based numbering races/recycles; name-string linkage; non-transactional multi-call flows.
NFR-6	Auditability	Who did what, when — for ticket changes, deletions and KPI adjustments.	PARTIAL — notes carry author/time; ticket field changes and deletions are unlogged.
NFR-7	Compatibility	Evergreen desktop and mobile browsers.	MET in practice (screenshot shows mobile Safari use); no formal browser matrix.

6. Critical review and findings

This section is the requested critique of the reviewed artefacts. Findings are grouped and rated: Critical (exploitable or data-corrupting), High (functional failure or serious risk), Medium (quality/UX debt), Low (cosmetic).

6.1 Security findings

ID	Severity	Finding	Impact / evidence
SEC-1	Critical	Supabase project URL, anon key and workspace UUID are hard-coded in the shipped HTML (source lines ~844–846).	Anyone who views source can construct direct REST calls against the production database.
SEC-2	Critical	Row Level Security is enabled but every table has an allow_all policy (USING true, WITH CHECK true).	Combined with SEC-1, the entire database — including all role passwords — is readable and writable by anyone on the internet, no login required. All in-app permission checks are cosmetic.
SEC-3	Critical	Passwords are stored in plain text, downloaded to every client at page load, and compared in browser JavaScript. Minimum length is 4; no logout, rate limiting, or hashing.	The manager password (full control) is exposed in every visitor's network tab. Shared member password means no individual accountability.
SEC-4	High	Site-admin passwords are rendered in clear text in the Manager Settings list.	Shoulder-surfing / screen-share leakage of live credentials.
SEC-5	High	User-supplied content (ticket names, descriptions, notes, member names) is interpolated into innerHTML without escaping throughout the render layer.	Stored XSS: a crafted ticket name or note executes script in every viewer's browser — and since the anon key is public (SEC-1), an attacker does not even need UI access to plant it.
SEC-6	Medium	No session management: no tokens, no expiry, no persistence.	Open tabs remain privileged indefinitely; conversely a refresh logs users out (usability cost without a security gain).

6.2 Data-integrity findings

ID	Severity	Finding	Impact
DATA-1	High	Ticket numbers are generated client-side as count+1 over existing tickets.	Concurrent submissions mint duplicate numbers; deletions cause number reuse (EST-118 can exist twice). Numbers are the human reference on site — collisions are costly.
DATA-2	High	People and sites are linked to tickets by NAME STRING (tickets.assignee, site_name, submitted_by; kpi_entries.member_name; notes.author), not by foreign key.	Renames rely on best-effort cascading PATCHes that miss notes and KPI entries; duplicate names silently merge two people's queues and scores.
DATA-3	Medium	Multi-step writes (workspace + members in setup; delete-then-insert opening points; unassign-then-delete member; notes-then-	A failure mid-sequence leaves inconsistent state with no rollback or retry.

ID	Severity	Finding	Impact
		ticket delete) are separate REST calls with no transaction.	
DATA-4	Medium	The KPI one-award-per-ticket guard checks only the local in-memory cache.	Two users marking the same ticket Done within one polling window can double-award points.
DATA-5	High	The setup SQL embedded in the app omits kpi_settings, kpi_entries and tender_notes; production Supabase (screenshot) contains all three.	Schema source-of-truth is lost; a fresh install from the in-app instructions produces a broken KPI module.
DATA-6	Medium	updated_at and KPI month attribution use the client's clock; saves are last-writer-wins with no version/conflict check.	A device with a wrong clock mis-dates records and mis-scores deadlines; concurrent edits silently overwrite each other.

6.3 Functional defects found in the source

ID	Severity	Defect	Location / detail
BUG-1	Medium	Read-only ticket modal prints the literal text "\${t.assigned 'nobody'}" instead of the assignee's name.	openModal(): the read-only branch is a quoted string inside the template literal, so the placeholder never interpolates (line ~1565).
BUG-2	High	Site Admin ticket rows are not clickable; openSiteModal()/saveSiteDeadline() are fully implemented but unreachable.	renderSiteAdminTickets() renders <tr> without onclick. The advertised "update date" capability (page subtitle) is dead — Site Admins cannot actually extend deadlines or view notes.
BUG-3	Low	KPI history badges for manual entries reference CSS class b-review, which is not defined.	renderKPIMonth()/renderMemberKPI(): manual-type badges render unstyled.
BUG-4	Low	Tautological condition curYear===curYear in the member KPI month highlight.	renderMemberKPI(): harmless today, but signals an intended year comparison that was never wired.
BUG-5	Medium	Auto-KPI failures are swallowed (console.error only).	autoKpiEntry(): ticket save reports success even when the points write failed.
BUG-6	Low	Member share links embed &ws=<id>, which the loader ignores.	copyMemberLink()/renderSettingsMembers() vs init(): dead parameter, misleading for multi-workspace ambitions.
BUG-7	Medium	"Setup new workspace" creates a workspace that can never be loaded.	doNewWorkspace() opens the wizard, but init() always loads the hard-coded workspace UUID, stranding the new workspace and its data.
BUG-8	Low	Config screen SQL and setup wizard are dead code in the production build.	Bypassed by hard-coded credentials; increases page weight and confuses maintenance.

6.4 Product and UX gaps

ID	Gap	Consequence
GAP-1	No free-text search anywhere (board or table).	At 118+ tickets, finding one ticket by name already requires scanning or browser Ctrl-F.
GAP-2	No pagination/virtualisation; full-dataset polling every 8 s per client; notes fetched via an in.() list of every ticket id.	Performance and URL-length ceilings will be hit as the register grows; bandwidth waste on mobile.
GAP-3	No drag-and-drop on the kanban board.	Status changes cost 3–4 clicks through the modal; the board metaphor under-delivers.
GAP-4	No notifications (e-mail / WhatsApp / push) on assignment, deadline approach or overdue.	Ironic given the intake sources tracked are WhatsApp/e-mail; members must poll the app themselves.
GAP-5	No file attachments on tickets.	Tender documents, drawings and BoQs live elsewhere; the ticket is not the single source of truth.
GAP-6	No KPI reversal/adjustment when tickets are reopened or deleted; no KPI export.	Ledger accuracy depends on manual bonus/penalty corrections by the manager.
GAP-7	KPI history limited to current + previous year (manager) and current year only (member).	No long-term performance trail; year-end reviews lose December visibility in January.
GAP-8	No audit log of ticket edits/deletions; no soft delete.	Disputes ("who moved my deadline?") cannot be resolved from data.
GAP-9	English-only UI for an Indonesian-speaking user base (ticket content is Bahasa Indonesia).	Adoption friction; mixed-language records.
GAP-10	No accessibility provisions (keyboard navigation, ARIA, contrast verification).	Excludes assistive-technology users; potential compliance issue.

6.5 Architecture assessment

Strengths: the single-file, zero-build, serverless approach delivered a genuinely useful production tool very fast; role separation is well thought through for the workflow; the fixed five-stage pipeline matches how estimation work actually moves; the KPI ledger design (typed transactions rather than mutable totals) is sound and auditable; visual language is consistent and the overdue treatment is excellent.

Weaknesses: the same choices that made it fast are now the main liabilities — client-side trust for everything (authorisation, numbering, scoring), name-string coupling instead of relational integrity, one 2,400-line file mixing markup, style, logic and configuration (no tests, no modules, no version control evident — the file is literally named "index (1).html"), and a data layer whose security posture is incompatible with holding commercial tender data.

6.6 Consistency review of the document set

- The reviewed source (EstimaPro, EST-NNN numbering, four work types, five statuses) does not match the deployed screenshot (EngPro, #EP-NNN numbering, a "Tender" type label, extra nav items PROGRAM and New Ticket, an Edit action per row).
- Production Supabase contains a tender_notes table the reviewed source never touches — confirming a newer forked build exists whose source was not provided.
- Screenshot branding "KPN Downstream-Estimation Control" confirms the workspace record differs from the code default "EstimaPro".

- **CONCLUSION:** this PRD should be validated against the current deployed build; Section 4 documents the provided source as the requirements baseline, and the fork must be reconciled into one codebase before further development (see R-10).

7. Recommendations and roadmap

7.1 Priority 0 — security remediation (do before anything else)

#	Recommendation
R-1	Replace shared plain-text passwords with Supabase Auth (email magic-link or username/password with hashing). One identity per person; remove password fields from the workspaces/site_admins tables.
R-2	Rewrite RLS policies to enforce workspace membership and role per table (drop every allow_all policy). The anon key may remain public only once RLS actually restricts it.
R-3	Escape all user-supplied content before insertion into the DOM (textContent or a sanitiser) to close stored XSS.
R-4	Remove site-admin password display from Settings; add rotate-password instead. Enforce a real password policy.

7.2 Priority 1 — data integrity

#	Recommendation
R-5	Generate ticket numbers server-side (Postgres sequence or trigger per workspace) to eliminate duplicates and reuse.
R-6	Migrate name-string links to foreign keys (tickets.assignee_id → members.id, tickets.site_id → site_admins.id, kpi_entries.member_id); backfill from existing names.
R-7	Move auto-KPI scoring to a database trigger or edge function on the status transition, with a UNIQUE (ticket_id, type='auto') constraint — closes both the double-award race and the silent-failure divergence, and add a reversal entry on reopen.
R-8	Maintain one canonical schema migration (SQL file in version control) covering all eight tables; delete the stale in-app SQL.

7.3 Priority 2 — product completion

#	Recommendation
R-9	Fix shipped defects: BUG-1 (read-only message), BUG-2 (wire Site Admin rows to openSiteModal — the feature is already built), BUG-3 (badge class), BUG-5 (surface KPI write failures), BUG-7 (workspace reset).
R-10	Reconcile the EngPro fork with this source into one repository with version control; adopt the deployed naming (#EP numbering, current nav) as canonical in PRD v1.1.
R-11	Replace 8 s full-refetch polling with Supabase Realtime subscriptions; add search, pagination and column sorting to the ticket list.
R-12	Add drag-and-drop status changes on the board with an undo toast.
R-13	Add notifications: assignment, T-2 days deadline warning, overdue digest (e-mail first; WhatsApp via provider later).
R-14	Add attachments (Supabase Storage) and an audit log (edit history per ticket, soft delete).
R-15	Add KPI export (CSV/XLSX) and unlimited year history; add Bahasa Indonesia UI option.

7.4 Suggested release plan

Release	Contents	Exit criteria
v1.1 (hardening)	R-1...R-4, R-9, R-10	Pen-test style check: no data readable/writable without authenticated, role-scoped session; all four roles function end-to-end; single repository.
v1.2 (integrity)	R-5...R-8	Concurrency test: parallel submissions produce unique numbers; rename/duplicate-name scenarios keep queues and KPI intact.
v2.0 (experience)	R-11...R-15	Realtime board < 1 s propagation; search returns in < 300 ms at 5,000 tickets; notification delivery verified.

8. Open questions

- Which build is canonical — this source file or the deployed EngPro variant (tender_notes, PROGRAM tab, #EP numbering)? The newer source was not in the /docs folder.
- Should Manager also have a ticket-intake form? (The code routes submitTicket for admins only; managers currently edit but cannot create from their screen.)
- Is KPI month attribution intended to follow completion date (current behaviour) or deadline month?
- Should tickets without deadlines be allowed, given they always score on-target points?
- Data-retention and privacy expectations for tender data (commercially sensitive) — who may hold the manager credential?

9. Appendix — screen inventory

Screen	Role	Pages / tabs
Config screen	(dormant)	Supabase URL/key entry + SQL instructions
Setup wizard	(dormant)	Company → Team → Passwords → Done
Login	All	Role selection → password (+ name/site select)
Manager app	Manager	Dashboard · Board · Tickets · KPI · Settings
Admin app	Admin	New request · Board · All tickets
Site Admin app	Site Admin	New request · My tickets
Member app	Member	My tickets · Team board · My KPI
Shared overlays	All	Ticket modal · confirmation dialogs · KPI modals · toast · loading overlay · sync indicator